

Object Oriented Analysis and Design

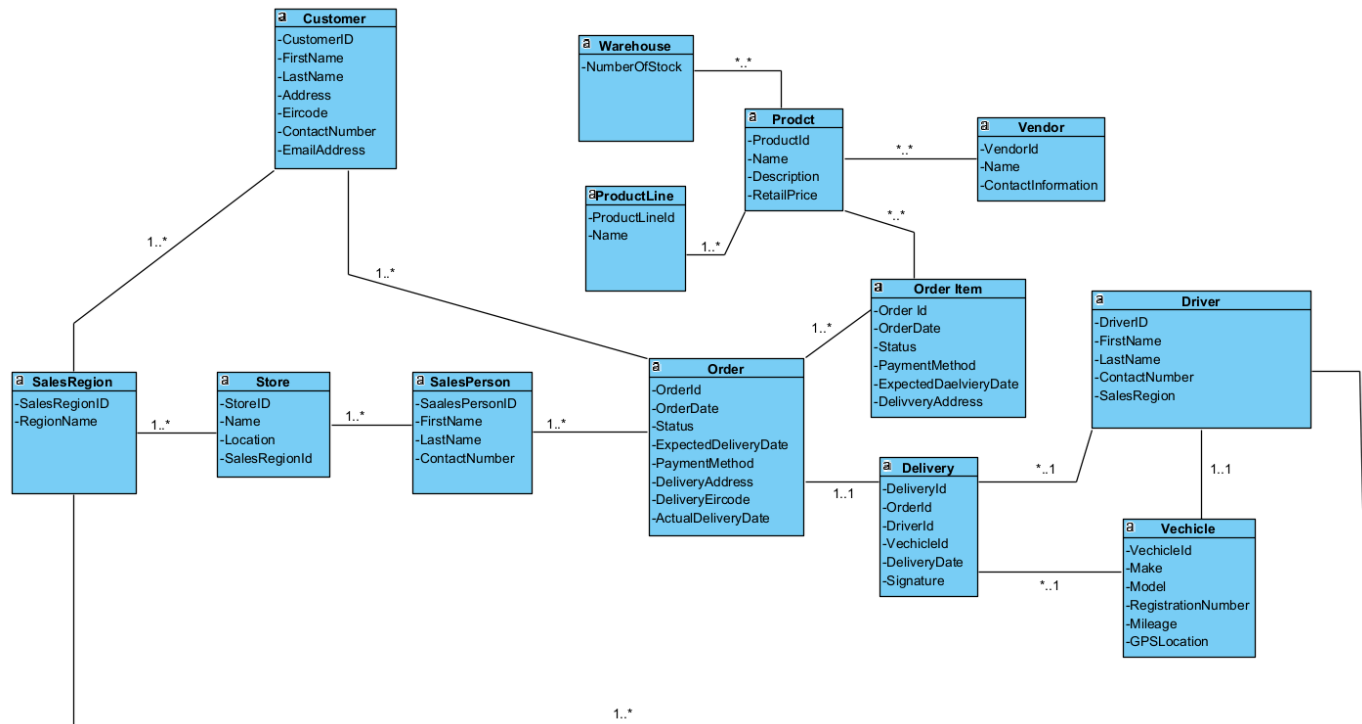
Brendan Daly & Rebecca Lordan

Brendan Daly & Rebecca Lordan.....	1
Table Of Content.....	2
Section A - Oak Valley Furniture Company.....	2
Question 1.....	2
Question 4.....	6
Question 5.....	9
Question 6.....	10
Section B - Design Pattern Related Questions.....	11
Question 1.....	11
Question 2.....	12
Student Declaration.....	14
Brendan Daly.....	14
Rebecca Lordan.....	15

Section A - Oak Valley Furniture Company

Question 1 Part A

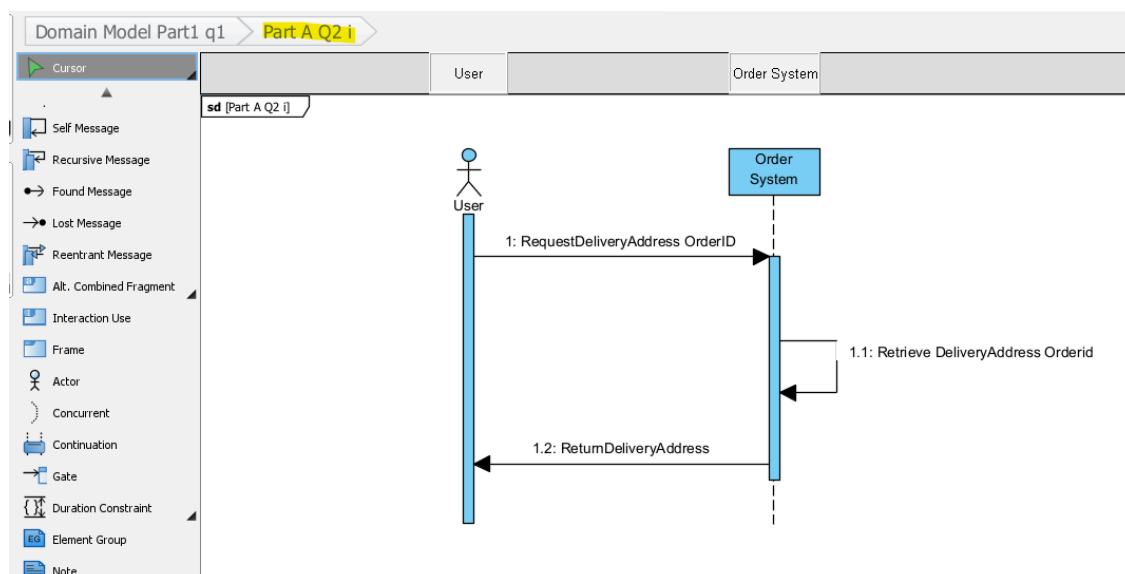
Q1. Create a Domain Model for Oak Valley



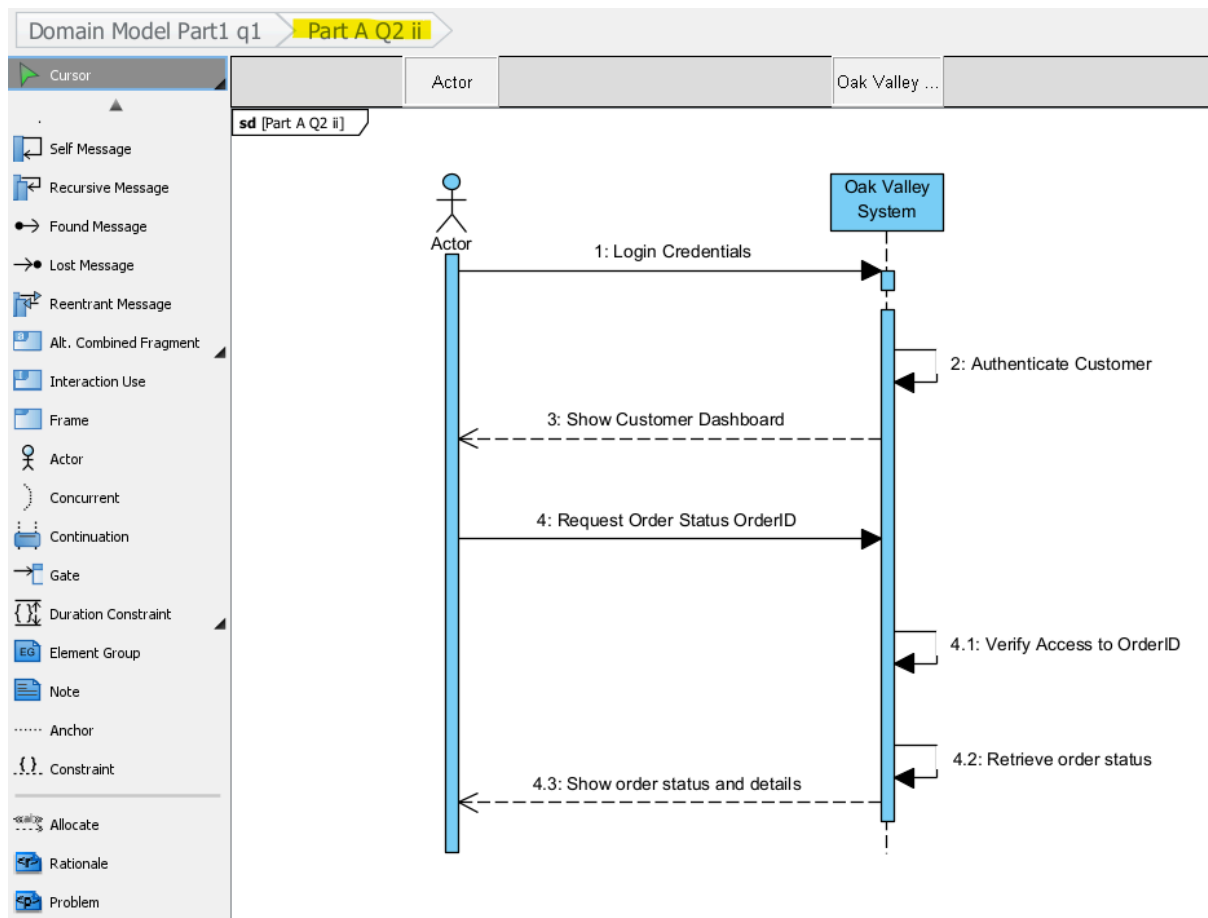
Question 2

Create system sequence diagrams to model the following features:

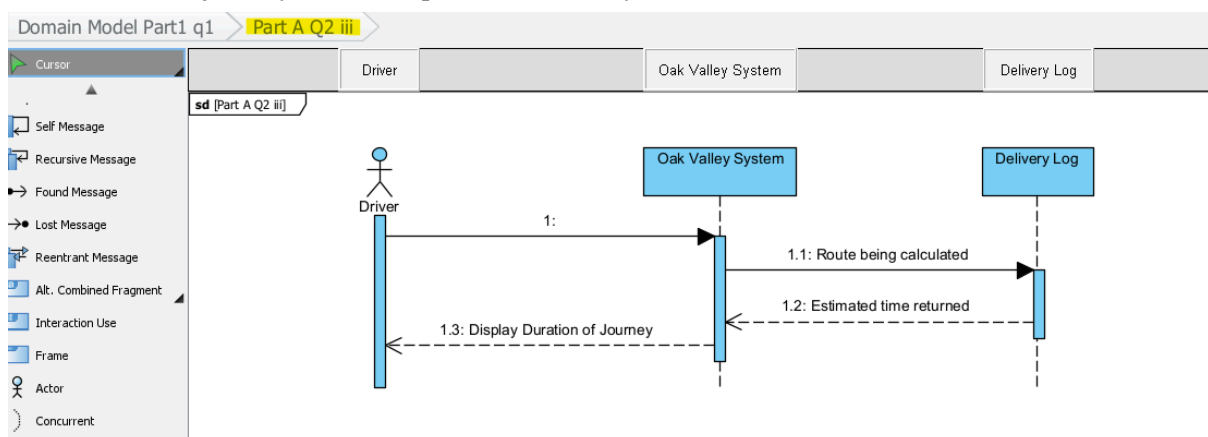
1. Find the delivery address of an order.



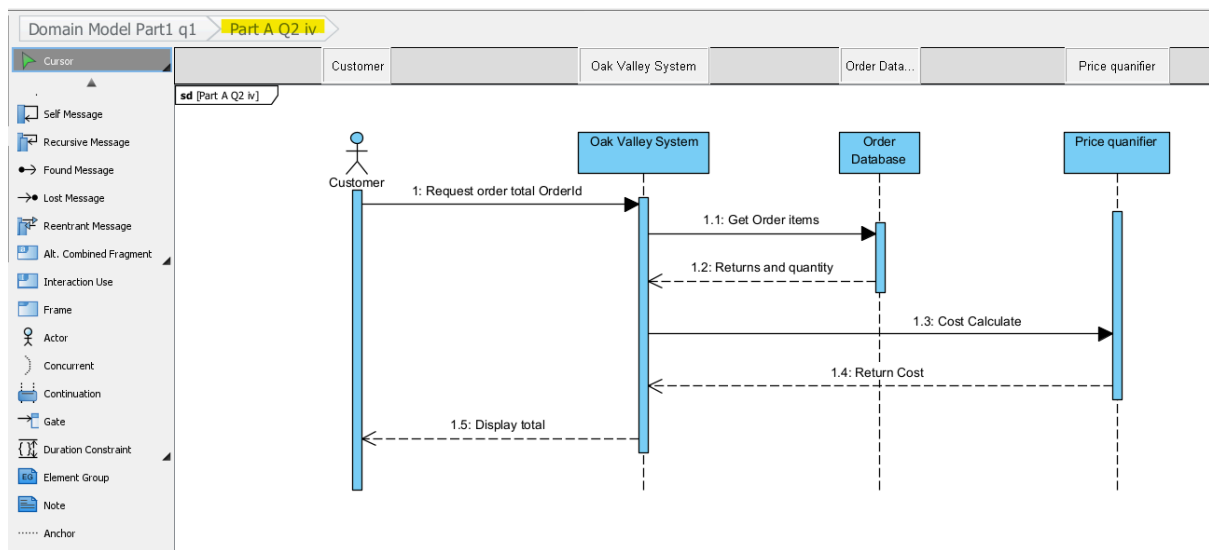
2. Find the status of an order.



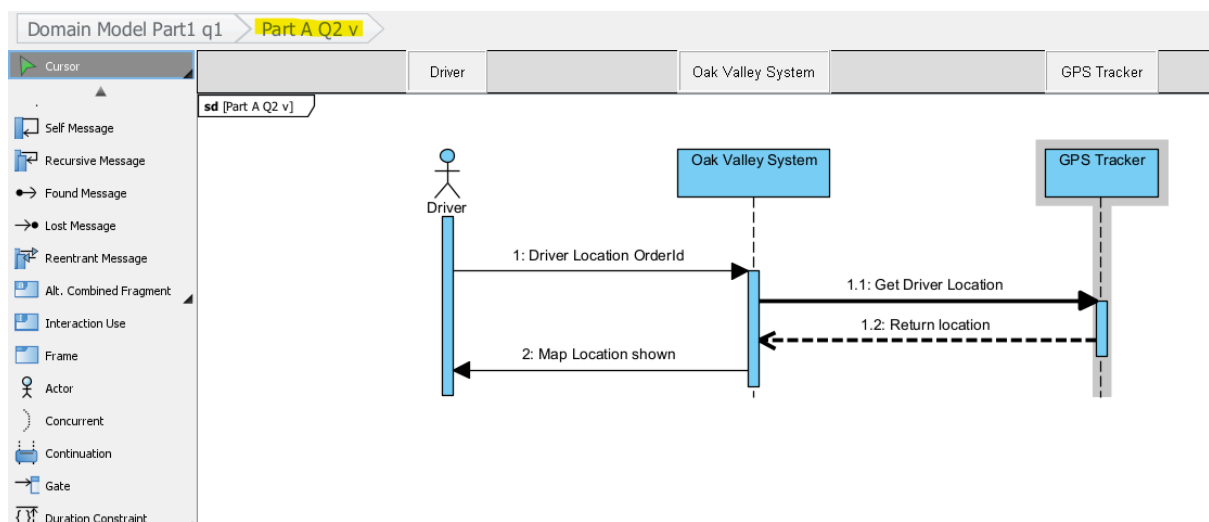
3. Find the journey time for a particular delivery schedule



4. Find the total cost of an order for a particular customer.



5. Find the current location of a driver who is out on a delivery of a customer order.



Question 3

Identify and list operations signatures required to complete each feature. Assign each operation to the appropriate class.

1. Find Delivery Address

OrderService - getDeliveryAddress(orderID)

OrderRepository - findOrder(orderID)

2. Check Order Status

OrderService - getOrderStatus(orderID)

OrderRepository - findOrder(orderID)

3. Calculate Order time

DeliveryService - calculateJourneyTime(deliveryID)

LogisticsSystem - getRoute(address)

4. Calculate Order Cost

OrderService - calculateTotalCost(orderID)

PricingService - getProductPrice(productID)

DiscountService - applyDiscounts(order)

5. Track Driver Location

DeliveryService - trackDriver(driverID)

FleetSystem - getVehicleLocation(vehicleID)

Question 4

Sequence Diagrams to realise the happy scenario

1. Find the Delivery Address of an Order

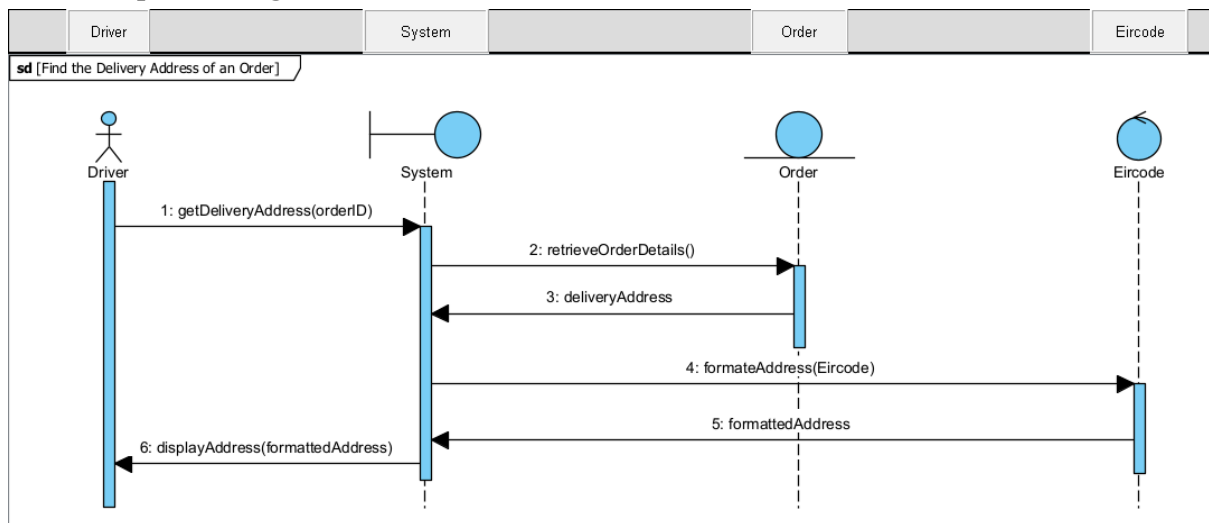
Use Case - Locate delivery address using Eircode

Actors - Driver, System, Order, Eircode

Happy Scenario

1. Driver looks for the delivery address of an order
2. System retrieves Eircode from order details
3. System displays formatted address with Eircode

Sequence Diagram



2. Find the Status of an Order

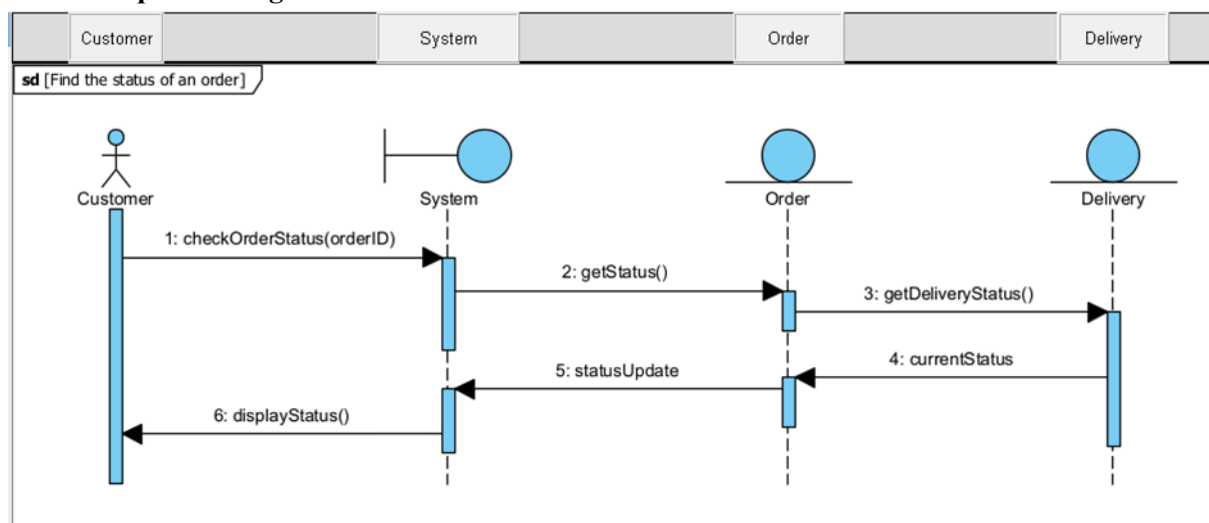
Use Case - Track order progress

Actors - Customer, System, Order, Delivery

Happy Scenario

1. Customer requests status update
2. System checks for the order and delivery status
3. System returns the current status of the order

Sequence Diagram



3. Find the Journey Time for a Particular Delivery Schedule

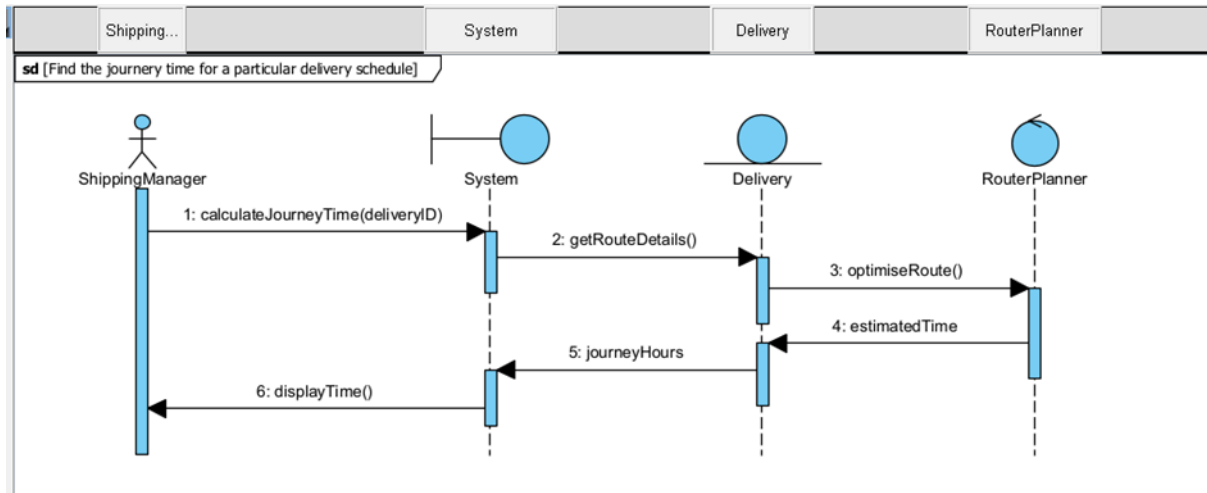
Use Case - Estimate delivery time

Actors - ShippingManager, System, Delivery, RoutePlanner

Happy Scenario

1. Manager requests the journey time
2. System calculates using GPS data and traffic patterns
3. System returns an estimated delivery time.

Sequence Diagram



4. Find the Total Cost of an Order for a Particular Customer

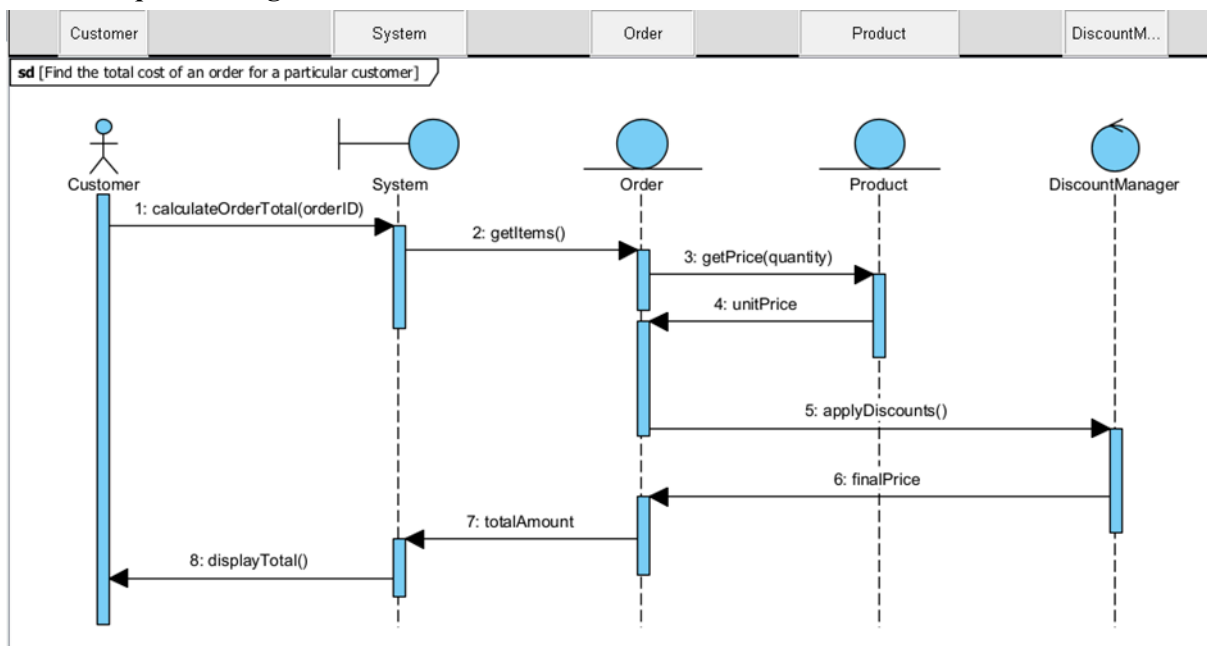
Use Case - Generate order Invoice

Actors - Customer, System, Order, Product, DiscountManager

Happy Scenario

1. Customer requests the total cost of an order
2. System calculates the product prices with discounts
3. System displays the total amount

Sequence Diagram



5. Find the Current Location of a Driver who is out on a Delivery of a Customer Order

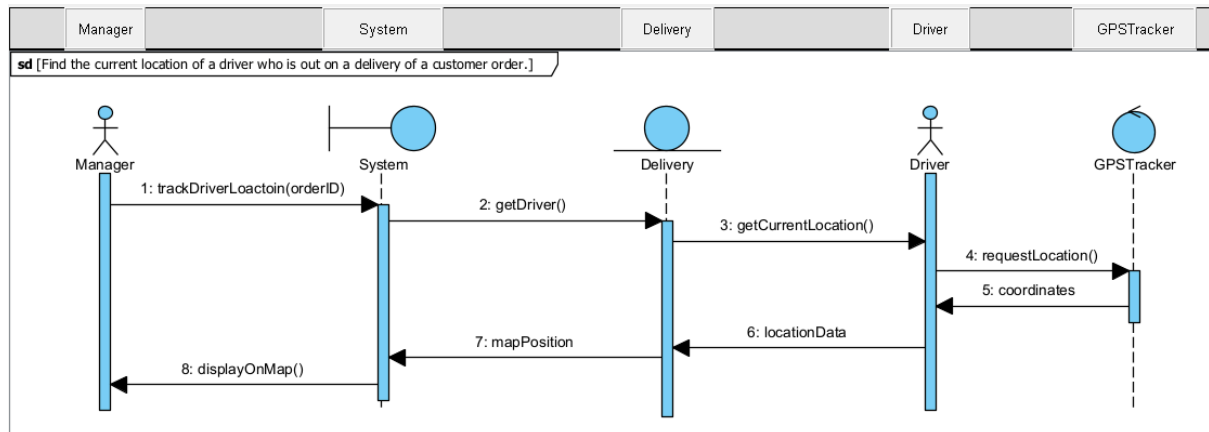
Use Case - Monitor delivery progress

Actors - Manager, System, Driver, GPSTracker

Happy Scenario

1. Manager requests a location update
2. System checks the GPS tracker
3. System displays real-time map position of the drivers location

Sequence Diagram



[Sequence Diagram Cheat Sheet. pdf - Canvas]

[Visual paradigm [Sequence Diagram](#)]

[Sequence Diagram [Sequence Diagrams – Unified Modeling Language \(UML\) | GeeksforGeeks](#)]

Question 5

Create a Design Class Diagram to support the sequence diagrams

System - Boundary Class, it's the main interface for user interactions.

Control Classes -

DeliveryController, used to manage delivery-related operations.

DiscountManager, it handles the application of discounts to orders.

GPS Tracker, it manages GPS tracking for drivers and vehicles.

Eircode, it handles the Eircodes.

Entity Classes -

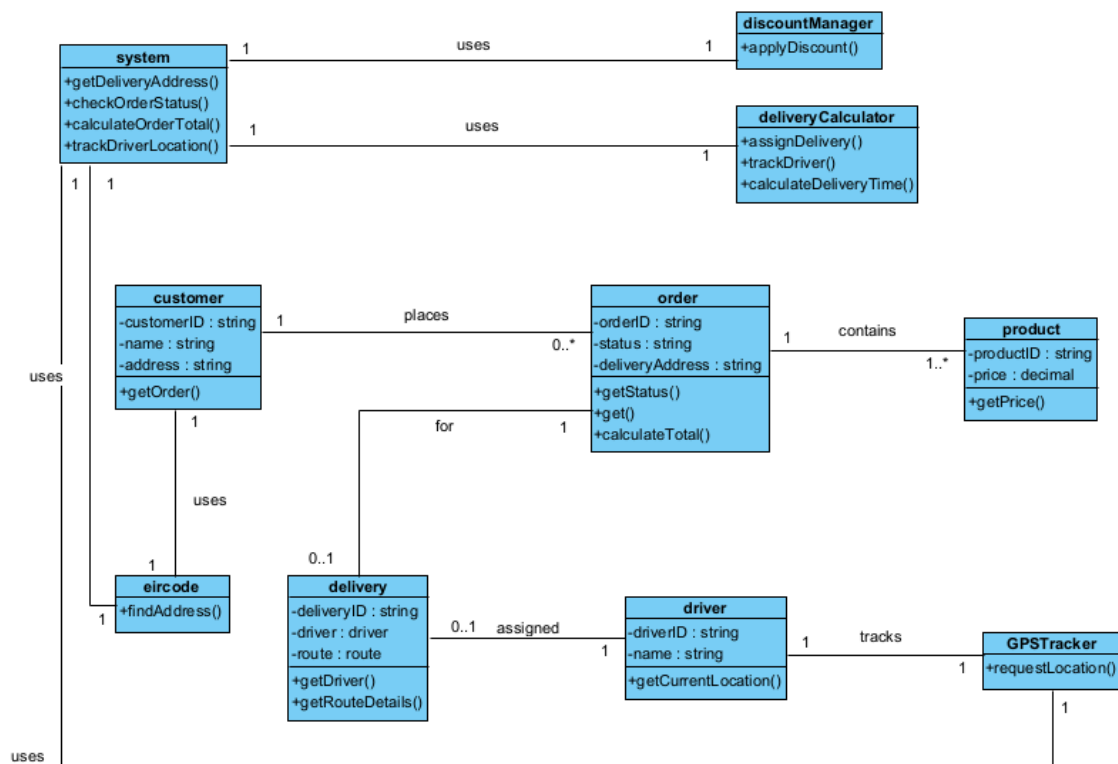
Order which represents the customer orders,

Customer which stores customer information,

Product which represents the products in the system,

Delivery which manages delivery information &

Driver which represents the delivery drivers.



[Class Diagram Cheat Sheet. pdf - Canvas]

[Visual Paradigm [UML Class Diagram Tutorial](#)]

[How to Draw a Class Diagram. pdf -Canvas]

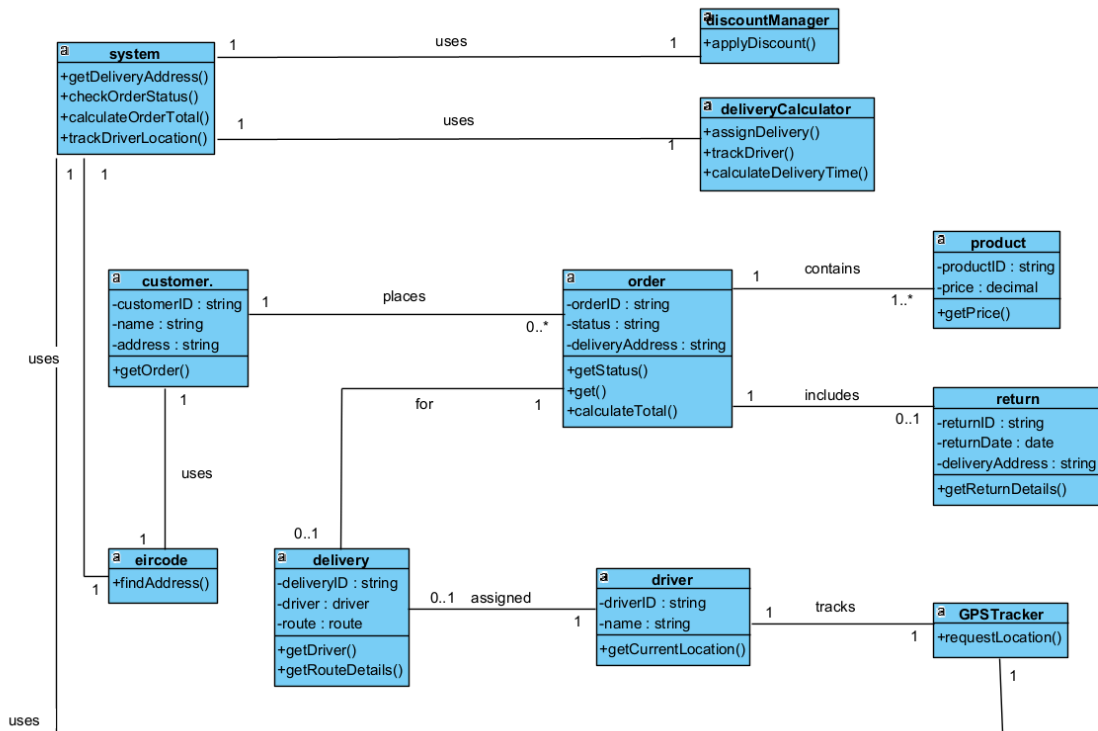
[Class Diagram [Class Diagram | Unified Modeling Language \(UML\) | GeeksforGeeks](#)]

Question 6

Extend design Class Diagram to allow for the following additional requirement.

i. A customer would like to return a product that was ordered and delivered.

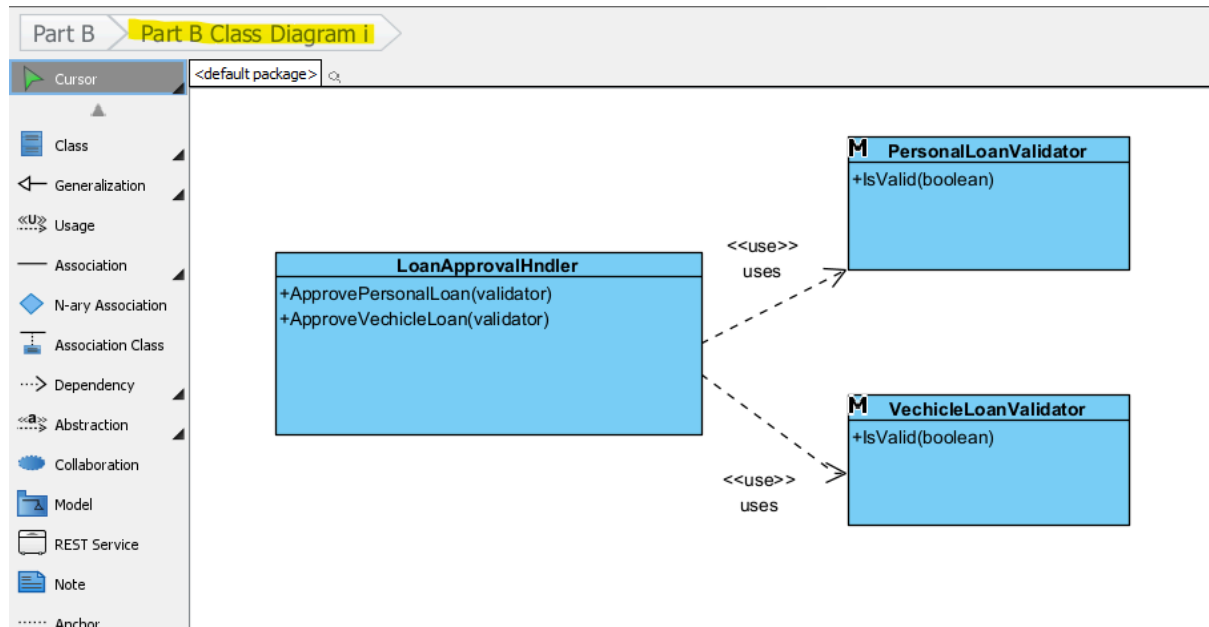
The date and reason for return needs to be tracked.



Section B - Design Pattern Related Questions

Question 1

i. Draw a class diagram to model the code



ii. Explain why the code violates the open closed principle

The code violates the open closed principle because every time you add a new loan type, you have to change the `LoanApprovalHndler` class (by adding a new method for the new loan).

This means the class is not "closed for modification." You should be able to add new loan types without changing existing code.

iii. Rewrite the code so it does not violate this design principle.

```
// LoanValidator

public interface LoanValidator {
    boolean isValid();
}

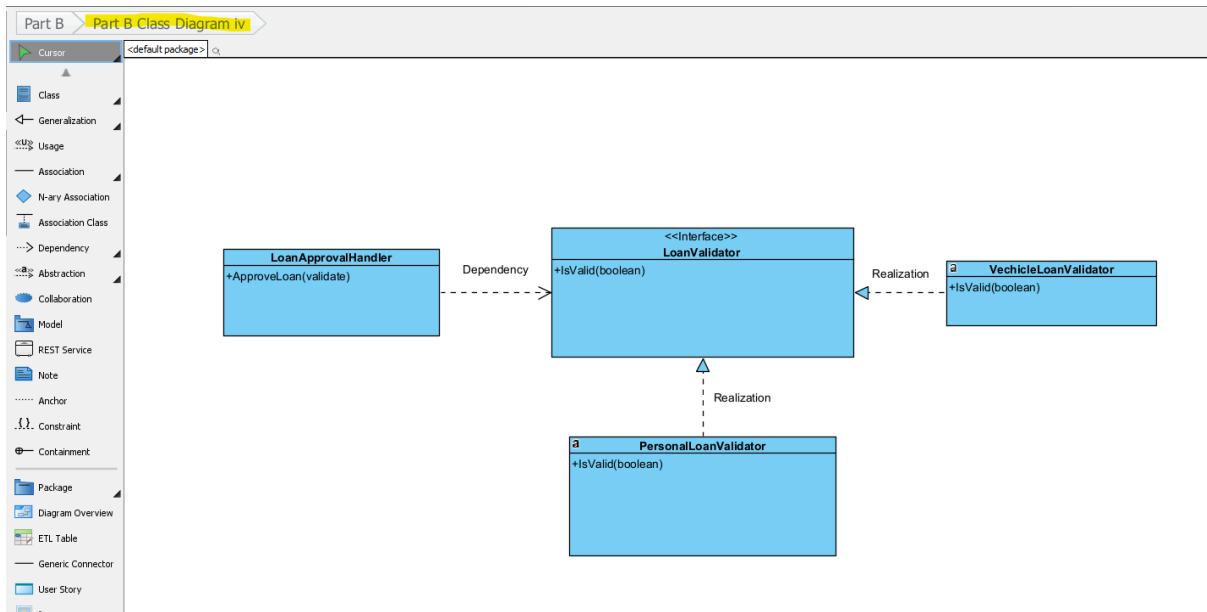
// Personal loan validator
public class PersonalLoanValidator implements LoanValidator {
    public boolean isValid() {
        // Validation logic
        return true;
    }
}

// Vehicle loan validator
public class VehicleLoanValidator implements LoanValidator {
    public boolean isValid() {
        // Validation logic
        return true;
    }
}

// Loan approval handler
public class LoanApprovalHandler {
    public void approveLoan(LoanValidator validator) {
        if (validator.isValid() {

        }
    }
}
```

iv. Draw a class diagram to model revised code.



Question 2

Design Class Diagram

Remote-Control - can control any device.

Device - is an interface implemented by the TV, Projector and SurroundSoundSystem.

TV

Project

SurroundSoundSystem

Person - interacts with the Remote-Control to control devices

Classes

Remote-Control - class is responsible for selecting a device and controlling it. It can connect to one device at a time and turn it on or off.

Its attributes are `currentDevice: Device` , which represents the device that is selected and controlled by the remote control.

Its methods are `choose_device(Device)` , which allows the user to choose and connect the remote to a device.

`turn on()` - Turns on the selected device.

turn off() - Turns off the selected device.

Device - class is an interface that all devices inherit from.

Its methods are `turn_on()` - Turns on the selected device.

turn off() - Turns off the selected device.

TV - class is an implementation of the Device class.

Its methods are `turn_on()` - Turns on the selected device.

turn off() - Turns off the selected device.

Projector - class is an implementation of the Device class.

Its methods are `turn_on()` - Turns on the selected device.

turn off() - Turns off the selected device.

SurroundSoundSystem - class is an implementation of the Device class.

Its methods are `turn_on()` - Turns on the selected device.

turn off() - Turns off the selected device.

Person - class that represents the user. The person uses the Remote-Control to interact with devices.

Its methods are `test()`

Relationships

Remote-Control and Device

Multiplicity One Remote-Control can control one Device at a time, but it can change devices.

Person and Remote-Control

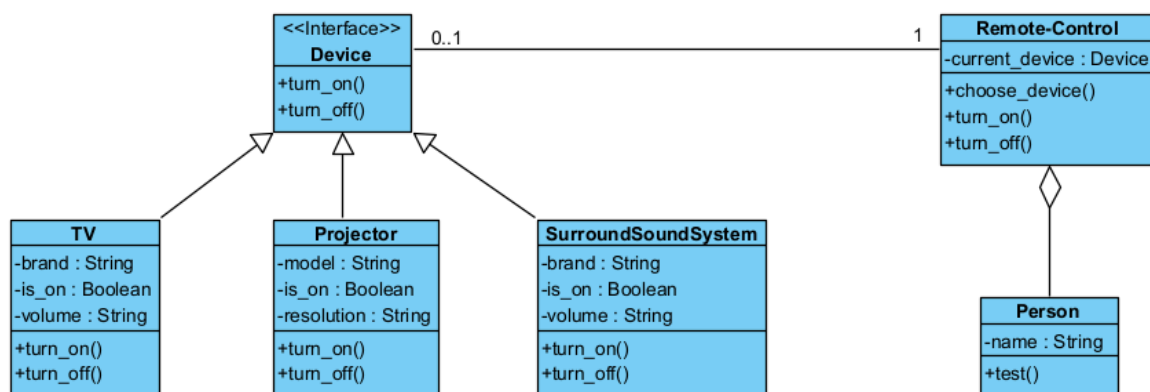
Multiplicity One Person uses one Remote Control.

Device and Person

Multiplicity A Person can control multiple devices (TV, Projector & SurroundSoundSystem), but only one at a time using the Remote-Control.

Device and Remote-Control

Multiplicity A Device can be controlled by one Remote-Control at a time.



[Class Diagram Cheat Sheet. pdf - Canvas]

[Visual Paradigm [UML Class Diagram Tutorial](#)]

[How to Draw a Class Diagram. pdf -Canvas]

[Class Diagram [Class Diagram | Unified Modeling Language \(UML\) | GeeksforGeeks](#)]

Student Declaration

Brendan Daly

I can confirm the following details:

Student ID/Registration number: R00108016

Name: Brendan Daly

Module Name: OO Analysis and Design

Lab Lecture: Nasir Ahmad

Due Date: 27th April 2025 @midnight

Rebecca Lordan

I can confirm the following details:

Student ID/Registration number: R00225669

Name: Rebecca Lordan

Module Name: OO Analysis and Design

Lab Lecture: Nasir Ahmad

Due Date: 27th April 2025 @midnight